

CS571: Programming Languages

Assignment 5. Solution

Problem 1 [8pt] Consider the following code snippet with exceptions. Note that the main function calls `foo` in the nested try block.

```
void foo () {
    try {
        throw new Exception1();
        print ("A");
        throw new Exception2();
        print ("B");
    }
    catch(Exception1 e1) {
        print "handler1";
    }
    print ("C");
    throw new Exception2();
}

void main () {
    try {
        try {
            foo();
            print ("D");
        }
        catch(Exception1 e1) { print "handler2"; }
    }
    catch(Exception2 e2) { print "handler3"; }
}
```

a) (4pt) Write down what will be printed out by the program and briefly justify your answer.

Solution: It will print out:

handler 1

C

handler 3

b) (4pt) Instead of the “replacement” semantics of exception handling used in modern languages (i.e., the semantics introduced in lecture), a very early design of exception handling introduced in the PL/I language uses a “binding” semantics. In particular, the design dynamically tracks a sequence of “catch” blocks that are currently active; a catch block is active whenever the corresponding try block is active. Moreover, whenever an exception is thrown, the sequence of active “catch” blocks will be traversed (in a first-in-last-out manner) to find a matching handler. Furthermore, execution will resume at the statement following the one that throws exception, rather than the next statement after the matching “catch” block as we have seen in the “replacement” semantics. Write down what will be printed out by the program if the language uses the “binding” semantics, and briefly justify your answer.

Solution: It will print out:

handler1

A

handler3

B

C
handler3
D

Problem 2 [8pt] For each of following terms, compute the set of free variables, then connect all bound variables to their definitions with lines. For example, the answer for $\lambda x. x x y$ should be $\{y\}$ and $\lambda \underline{x. x} x y$.

For this problem a solution without detailed steps or explanation will receive ZERO points.

a) (4pt) $\lambda x. y x \lambda x. x y$

Solution: $\lambda \underline{x. y x} \lambda \underline{x. x} y$

$$\begin{aligned}
 \text{FV}(\lambda x. y_1 x_1 \lambda x. x_2 y_2) &= \text{FV}(y_1 x_1 \lambda x. x_2 y_2) - \{x\} \\
 &= \text{FV}(y_1 x_1) \cup \text{FV}(\lambda x. x_2 y_2) - \{x\} \\
 &= \text{FV}(y_1) \cup \text{FV}(x_1) \cup (\text{FV}(x_2 y_2) - \{x\}) - \{x\} \\
 &= \text{FV}(y_1) \cup \text{FV}(x_1) \cup (\text{FV}(x_2) \cup \text{FV}(y_2) - \{x\}) - \{x\} \\
 &= \{y_1\} \cup \{x_1\} \cup (\{x_2\} \cup \{y_2\} - \{x\}) - \{x\} \\
 &= \{x_1, y_1\} \cup (\{x_2, y_2\} - \{x\}) - \{x\} \\
 &\quad (x_2 \text{ is bound to the second } \lambda x) \\
 &= \{x_1, y_1\} \cup \{y_2\} - \{x\} \\
 &= \{x_1, y_1, y_2\} - \{x\} \\
 &\quad (x_1 \text{ is bound to the first } \lambda x) \\
 &= \{y_1, y_2\} \text{ (or } = \{y\} \text{)}
 \end{aligned}$$

b) (4pt) $\lambda y. \lambda z. z y \lambda y. x$

Solution: $\lambda y. \lambda \underline{z. z} y \lambda y. x$

$$\begin{aligned}
 \text{FV}(\lambda y. \lambda z. z y \lambda y. x) &= \text{FV}(\lambda z. z y \lambda y. x) - \{y\} \\
 &= (\text{FV}(z y \lambda y. x) - \{z\}) - \{y\} \\
 &= (\text{FV}(z y) \cup \text{FV}(\lambda y. x) - \{z\}) - \{y\} \\
 &= (\text{FV}(z) \cup \text{FV}(y) \cup (\text{FV}(x) - \{y\}) - \{z\}) - \{y\} \\
 &= (\{z\} \cup \{y\} \cup (\{x\} - \{y\}) - \{z\}) - \{y\} \\
 &\quad (\text{nothing is bound to the second } \lambda y) \\
 &= (\{z\} \cup \{y\} \cup \{x\} - \{z\}) - \{y\} \\
 &= (\{x, y, z\} - \{z\}) - \{y\} \\
 &\quad (z \text{ is bound to } \lambda z) \\
 &= \{x, y\} - \{y\} \\
 &\quad (y \text{ is bound to the first } \lambda y) \\
 &= \{x\}
 \end{aligned}$$

Problem 3 [8pt] Fully evaluate the following λ -term so that no further β -reduction is possible. You must show *all* the steps to earn credit for this problem.

a) (4pt) $((\lambda x y. x y) (\lambda x. y)) u$

Solution:

$$\begin{aligned} & ((\lambda x \ y. \ x \ y) (\lambda x. \ y)) \ u \\ = & ((\lambda x. (\lambda y. \ x \ y)) (\lambda x. \ y)) \ u && (\text{desugar term } \lambda x \ y. \ x \ y) \\ = & ((\lambda x. (\lambda z. \ x \ z)) (\lambda x. \ y)) \ u && (\alpha - \text{reduction}) \\ = & (\lambda z. (\lambda x. \ y) \ z) \ u && (\beta - \text{reduction}) \\ = & (\lambda x. \ y) \ u && (\beta - \text{reduction}) \\ = & y && (\beta - \text{reduction}) \end{aligned}$$

b) (4pt) $(\lambda x. \lambda y. \ y \ x) (\lambda z. \ y)$

Solution:

$$\begin{aligned} & (\lambda x. (\lambda y. \ y \ x)) (\lambda z. \ y) \\ = & (\lambda x. (\lambda u. \ u \ x)) (\lambda z. \ y) && (\alpha - \text{reduction}) \\ = & \lambda u. \ u \ \lambda z. \ y && (\beta - \text{reduction}) \end{aligned}$$

Problem 4 [8pt] Recall that under Church encoding, we have the following definitions:

$$\text{IF} \triangleq \lambda b \ t \ f. \ b \ t \ f \quad \text{TRUE} \triangleq \lambda t \ f. \ t \quad \text{FALSE} \triangleq \lambda t \ f. \ f$$

a) (4pt) Fully evaluate $(\lambda x. (x \ y \ \text{TRUE})) \ \text{FALSE}$ so that no further β -reduction is possible..

Solution:

$$\begin{aligned} & (\lambda x. (x \ y \ \text{TRUE})) \ \text{FALSE} \\ &= (\text{FALSE} \ y \ \text{TRUE}) && (\beta - \text{reduction}) \\ &= (\lambda t \ f. \ f) \ y \ \text{TRUE} && (\text{definition of FALSE}) \\ &= \text{TRUE} && (\beta - \text{reduction}) \end{aligned}$$

b) (4pt) Define XOR using λ -calculus and IF, TRUE, FALSE as defined above. Show that with your definition, XOR FALSE TRUE fully evaluates to TRUE. And that XOR TRUE TRUE fully evaluates to FALSE. Show every step of the evaluation.

Solution:

$$\text{XOR} \triangleq \lambda b_1 \ b_2. \ b_1 \ (b_2 \ \text{FALSE} \ \text{TRUE}) \ b_2$$

$$\begin{aligned} & (\text{XOR} \ \text{FALSE} \ \text{TRUE}) \\ &= (\lambda b_1 \ b_2. \ b_1 \ (b_2 \ \text{FALSE} \ \text{TRUE}) \ b_2) \ \text{FALSE} \ \text{TRUE} && (\text{definition of XOR}) \\ &= \text{FALSE} \ (\text{TRUE} \ \text{FALSE} \ \text{TRUE}) \ \text{TRUE} && (\beta - \text{reduction}) \\ &= (\lambda t \ f. \ f) \ (\text{TRUE} \ \text{FALSE} \ \text{TRUE}) \ \text{TRUE} && (\text{definition of FALSE}) \\ &= \text{TRUE} && (\beta - \text{reduction}) \end{aligned}$$

$$\begin{aligned} & (\text{XOR} \ \text{TRUE} \ \text{TRUE}) \\ &= (\lambda b_1 \ b_2. \ b_1 \ (b_2 \ \text{FALSE} \ \text{TRUE}) \ b_2) \ \text{TRUE} \ \text{TRUE} && (\text{definition of XOR}) \\ &= \text{TRUE} \ (\text{TRUE} \ \text{FALSE} \ \text{TRUE}) \ \text{TRUE} && (\beta - \text{reduction}) \\ &= (\lambda t \ f. \ t) \ (\text{TRUE} \ \text{FALSE} \ \text{TRUE}) \ \text{TRUE} && (\text{definition of TRUE}) \\ &= \text{TRUE} \ \text{FALSE} \ \text{TRUE} && (\beta - \text{reduction}) \\ &= (\lambda t \ f. \ t) \ \text{FALSE} \ \text{TRUE} && (\text{definition of TRUE}) \\ &= \text{FALSE} && (\beta - \text{reduction}) \end{aligned}$$

Problem 5 [8 + (2 bonus) pt] In the lecture, we have used the pure λ -calculus to construct natural numbers and encoded some operations on them. In such encoding, a number n is encoded as a function $\lambda f \ z. \ f^n \ z$ (denoted as $\underline{n}$). In this problem, we will define more useful operations on numbers. If you get stuck in any question, try to proceed by assuming the previous function is properly defined. You can reuse any encoding given in lectures and previous questions.

a) (4pt) Define a function ISZERO in λ -calculus, so that given a number $\underline{n}$, it returns TRUE (the encoding of true) if $\underline{n} = \underline{0}$; FALSE (the encoding of false) if $\underline{n} \neq \underline{0}$. Hint: try to define two terms so that applying the second term multiple times to the first term returns FALSE; otherwise, the application returns TRUE.

Solution: $\text{ISZERO} \triangleq \lambda n. \ n \ (\lambda f. \ \text{FALSE}) \ (\text{TRUE})$

- b) (2pt **bonus**) Define a function PRED in λ -calculus, so that given a number $\underline{n}$, the function returns its predecessor, assuming the predecessor of $\underline{0}$ is $\underline{0}$. Hint: You can assume the following operations are given below (hence, you don't need to define them): PAIR constructs a pair of values, LEFT/RIGHT returns the first/second element of a pair. For example, LEFT (PAIR x y) evaluates to x .

$$\text{PAIR} \triangleq \lambda x y f.f x y \quad \text{LEFT} \triangleq \lambda p.p \lambda x y.x \quad \text{RIGHT} \triangleq \lambda p.p \lambda x y.y$$

Solution: $\text{PRED} \triangleq \lambda n. \text{RIGHT} (n (\lambda p. \text{PAIR} (\text{SUCC} (\text{LEFT } p)) (\text{LEFT } p)) (\text{PAIR } \underline{0} \underline{0}))$

- c) (4pt) Assume PRED in part b) is defined, use it to define a subtraction function MINUS, so that MINUS $\underline{n_1} \underline{n_2}$ returns $\underline{n_1 - n_2}$ when $n_1 \geq n_2$, and $\underline{0}$ otherwise.

Solution: $\text{MINUS} \triangleq \lambda n_1 n_2. n_2 \text{PRED } n_1$

Bonus

Problem 6 [2pt] In this problem we will learn good practice to program code allocating resources that will handle exceptions correctly, i.e., will return the resource properly even in the event of an exception.

Explain why the use of `lock_guard` is preferred over explicit mutex lock and unlock. You can read more about `lock_guard` from here: https://en.cppreference.com/w/cpp/thread/lock_guard.

Bad style using mutex lock and unlock:

```
std::mutex mtx;

void func()
{
    mtx.lock();
    // do things
    mtx.unlock();
}
```

Good style using `lock_guard`:

```
std::mutex mtx;

void func()
{
    std::lock_guard<std::mutex> lock(mtx);
    // do things
}
```

Solution: Things could go wrong and the control flow could abruptly skip the `mtx.unlock()` for all different reasons (exceptions thrown, premature return from the function and so on) in the bad-style implementation, and keeping the mutex locked and eventually creating unexpected deadlock.

The second approach takes advantage of the RAII style feature where the `lock_guard` lock will automatically reclaim resources (unlock the mutex lock it is associated with) in its destructor which is guaranteed to be called when lock goes out of scope. It doesn't matter how the control flow gets out of the function `func()`, through exceptions, premature returns or normal completion etc.